

LAMBERT-PROX-CLOSE1

Formalisation consolidee et pack autonome

Primitive de Wright Omega en deux coordonnees, sept operateurs
proximaux, PDHG-TV periodique, diagnostics gap/KKT, noyaux Triton et
routage local Tesla T4

Statut	CLOSE1 ferme; attestation Tesla T4 PASS 47/47
Auteur du projet	Laurent Hamza
Date	31 juillet 2026
Archive incorporee	LAMBERT_PROX_CLOSE1_AUTONOME_2026-07-31.zip
SHA-256	723e41c0cc78ab7cf180a2a251990b5ebc6b179ba2d46b9976a6271ddab1195c

Formalisation, audit, implementation et validation assistes par IA. Ce document separe explicitement theoremes exacts, diagnostics numeriques, preuves historiques, attestation T4 et limites de portee.

Statut executif

Le noyau proximal et la chaine NumPy - Torch - Triton - PDHG - routage restent valides sur leur perimetre publie. CLOSE1 corrige quatre points sans reinterpreter les donnees : la norme finie de K, le vocabulaire d'arret, l'absence de gap/KKT et l'auto-contenance du livrable.

Objet	Statut	Preuve ou limite
Wright Omega / U(R)	ACQUIS	$U + \log U = R$; implementation bi-coordonnee testee
Sept prox	ACQUIS	Formules et domaines testes; $x \log x$ ferme sur $x \geq 0$
Norme periodique	CORRIGEE	Formule exacte H,W; $\sqrt{8}$ seulement borne uniforme
PDHG exact	THEOREME	Condition $\tau \sigma \ K\ ^2 < 1$ dans le cadre convexe exact
Arret numerique	SEPARE	stabilized != certified != preuve de convergence flottante
CPU CLOSE1	PASS	58 tests; 5/5 modeles stabilises et certifies
T4 historique	PASS	47/47 correction; 10/10 lignes de routage; liaison post-run
T4 CLOSE1	PASS	47/47; hashes avant/apres identiques; sources locales concordantes
PDF seul	AUTONOME	Archive complete incorporee et verifiee par SHA-256

Verdict sobre : la contribution defendable est une architecture numerique integree et reproductible. Aucun nouveau theoreme proximal, aucun claim SOTA et aucune equivalence inter-GPU ne sont etablis.

1. Objet mathematique et statuts

Pour une fonction propre, convexe et semi-continue inferieurement f, l'operateur proximal est defini par :

$$\text{prox}_{(\lambda f)}(v) = \operatorname{argmin}_x \{ f(x) + \|x-v\|^2 / (2 \lambda) \}, \lambda > 0.$$

La primitive centrale resout $u + \log(u) = R$ avec $u > 0$. Elle est la fonction de Wright Omega : $U(R) = \Omega(R) = W_0(\exp(R))$. Le travail propre au corpus n'est donc pas l'invention de cette fonction, mais son contrat bi-coordonne ($u, q=\log u$), son evaluation robuste et son emploi commun dans plusieurs backends.

Derivees exactes

$$U'(R) = U/(1+U), \quad U''(R) = U/(1+U)^3.$$

La coordonnee q evite de reconstruire $\exp(R)$ lorsque R est grand et conserve une sortie exploitable lorsque u sous-deborde. Le statut OK_LOG_ONLY est donc une propriete du contrat logiciel, pas une approximation mathematique de la racine.

2. Catalogue proximal ferme

Fidelite $f(x)$	Domaine ferme	Prox ou reduction stable
$\exp(x)$	$\mathbb{R}$	$x = v - U(\log(\lambda)+v)$
$x \log x$	$x \geq 0; 0 \log 0 = 0$	$x = \lambda U(v/\lambda - \log(\lambda) - 1)$
$KL(x y)$	$x \geq 0, y \geq 0$	$y=0 \Rightarrow x=0$; sinon reduction a U
$\exp(x) - yx$	$x \in \mathbb{R}, y \geq 0$	$d=v+\lambda y; x=q-\log(\lambda)$
$x - y \log x$	$x \geq 0, y \geq 0$	racine quadratique conjuguee
$-\log x$	$x > 0$	racine quadratique conjuguee
$(x-y)^2/2$	$\mathbb{R}$	$x=(v+\lambda y)/(1+\lambda)$

Correction CLOSE1 : le domaine de $x \log x$ n'est pas $x > 0$. Sa fermeture convexe naturelle contient $x=0$ avec la convention $0 \log 0 = 0$. Le proximal, lui, est strictement positif pour les parametres finis testes.

3. Norme exacte du gradient periodique

Sur une grille $H \times W$, K est le gradient en differences avant periodiques. La transformee de Fourier discrete diagonalise chaque translation. Pour les frequences (k,l) , le symbole satisfait :

$$|K_{\text{hat}}(k,l)|^2 = 4 \sin^2(\pi k/W) + 4 \sin^2(\pi l/H).$$

Le maximum discret est atteint en $k=\text{floor}(W/2)$, $l=\text{floor}(H/2)$, d'ou :

$$||K||^2 = 4 \sin^2(\pi \text{floor}(W/2)/W) + 4 \sin^2(\pi \text{floor}(H/2)/H) \leq 8.$$

Pour 31×47 , CLOSE1 calcule $||K|| = 2.8258224296707$, contre $\sqrt{8} = 2.8284271247462$.

L'ancienne valeur etait donc une borne sure mais improprement nommee norme exacte. Le default ne rendait pas les pas instables; il les rendait legerement conservateurs.

Grille	Norme exacte	$\sqrt{8}$	Consequence
32×48	2.828427124746	2.828427124746	egalite
31×47	2.825822429671	2.828427124746	borne conservatrice

4. PDHG exact et implementation

Le probleme traite est $\min_x F(x; y_{\text{obs}}) + G(Kx)$, avec $G(z) = \lambda_{\text{TV}} \sum_i ||z_i||_2$. Son dual de Fenchel est $\max_p -F^*(-K^T p)$ sous $||p_i||_2 \leq \lambda_{\text{TV}}$.

$$\begin{aligned} p^{(n+1)} &= \text{proj}_B(p^n + \sigma K x_{\text{bar}}^n) \\ x^{(n+1)} &= \text{prox}_{(\tau F)}(x^n - \tau K^T p^{(n+1)}) \\ x_{\text{bar}}^{(n+1)} &= x^{(n+1)} + \theta (x^{(n+1)} - x^n) \end{aligned}$$

Theoreme exact utilise : pour des prox exacts et $\tau \sigma ||K||^2 < 1$, le schema de Chambolle-Pock converge sous les hypotheses convexes usuelles. Ce theoreme ne transforme pas automatiquement six iterations de Halley FP32 en prox exact. CLOSE1 conserve donc le theoreme et l'execution dans deux colonnes de statut differentes.

Niveau	Objet	Statut permis
Mathematique	PDHG a prox exacts	Convergence sous hypotheses et condition de pas
Torch FP64/FP32	Prox numeriques controles	Validation empirique + residus, pas identite exacte
Triton FP32	Halley fixe et noyaux fusionnees	Equivalence locale mesuree sur T4 historique
Arret	Petits changements d'iteres	Stabilisation seulement

Niveau	Objet	Statut permis
CLOSE1	Gap + KKT finaux	Certification numerique au seuil annonce

5. Conjugues et gap primal-dual

F(x)	F*(s)	Domaine dual
$(x-y)^2/2$	$s^2/2 + sy$	$s \in \mathbb{R}$
$x-y \log x$	$y[\log y - \log(1-s) - 1]$	$s < 1$ si $y > 0$; $s \leq 1$ si $y = 0$
$\exp x - yx$	$t \log t - t, t = s + y$	$t \geq 0$
$KL(x y)$	$y(\exp s - 1)$	$s \in \mathbb{R}, y > 0$
$x \log x$	$\exp(s - 1)$	$s \in \mathbb{R}$

Si p est dualement faisable, le gap calcule est :

$$\text{Gap}(x, p) = F(x) + \lambda_{TV} TV(x) + F^*(-K^T p) \geq 0.$$

Le gap relatif est $|\text{Gap}| / \max(1, |P|, |D|)$. La valeur absolue protege le test contre un minuscule gap negatif cause par l'arrondi; un gap materiellement negatif reste visible dans la valeur brute et invalide la finitude/acceptation attendue.

6. Residus KKT de point fixe

Les inclusions d'optimalite $0 \in \partial F(x) + K^T p$ et $p \in \partial G(Kx)$ sont testees par leurs resolvantes. Avec les pas utilises par le solveur :

$$\begin{aligned} r_x &= \|x - \text{prox}_{(\tau F)}(x - \tau K^T p)\| / (\tau S_x) \\ r_p &= \|p - \text{prox}_{(\sigma G^*)}(p + \sigma Kx)\| / (\sigma S_p) \end{aligned}$$

$S_x = \max(1, \|x\|, \|K^T p\|)$ et $S_p = \max(1, \|p\|, \|Kx\|)$. La certification numerique CLOSE1 exige simultanement : finitude, $\max(r_x, r_p) \leq \text{tol_KKT}$, $\text{gap_relatif} \leq \text{tol_gap}$ et violation de la boule duale $\leq \text{tol_KKT}$.

Champ	Signification	Ce qu'il ne prouve pas
stabilized	Critere de changement relatif atteint	Optimalite ou convergence flottante
certified	Gap et KKT passent les seuils demandes	Theoreme d'erreur a posteriori universel
status_note	Rappel textuel attache a chaque run	Extension du claim GPU

7. Resultats CPU CLOSE1

Environnement : Python 3.12.13, Torch 2.13.0+cpu, CPU. Suite complete :

..... [100%] 58 passed in 2.91s. Oracle NumPy : [100%] 10 passed in 0.15s.

Modele	Iter.	Baisse objectif	Gap relatif	KKT max	Statut
gaussian	112	46.2428	6.087e-11	4.267e-09	STAB + CERT
poisson_intensity	109	9.27566	3.639e-12	4.648e-09	STAB + CERT
poisson_log	89	13.4512	5.257e-12	6.950e-09	STAB + CERT
kl	132	9.33375	1.160e-10	4.337e-09	STAB + CERT
xlogx	40	536	0.000e+00	8.766e-12	STAB + CERT

Validation de la primitive Torch : 110008 points par dtype; residu normalise maximal FP64 3.331e-16, FP32 1.788e-07. Le gap d'adjoint periodique FP64 vaut 2.402e-15 dans le run PDHG local.

8. Preuves T4 : conservation et attestation CLOSE1

Le rapport historique T4 est conserve byte pour byte : 47/47 comparaisons PASS, pire ecart mis a l'echelle 4.768e-07. La relance CLOSE1 retourne 47/47 PASS, pire ecart mis a l'echelle 4.768e-07, environnement Torch 2.11.0+cu128 / CUDA 12.8 / Tesla T4.

Preuve	T4 historique	T4 CLOSE1
Correction noyaux	47/47 PASS	47/47 PASS
Routage	10/10 lignes promues	Table historique conservee
Hashes sources	Liaison post-run	Avant = apres = pack local
Statut	Valide sur perimetre 30 juillet	Atteste le 31 juillet

Le runner CLOSE1 a calcule la carte SHA-256 avant l'import du validateur CUDA, execute les 47 comparaisons, puis recalcule la meme carte. Le digest de release 7e0757258621520664c4...133172d2417f concorde exactement avec les huit sources du pack. Cette attestation ferme la provenance sur la Tesla T4 enregistree; elle ne s'etend pas a un autre GPU ou environnement.

9. Architecture de preservation

Modifier pdhg_promoted_v2.py aurait invalide les hashes du manifeste de promotion et ceux du benchmark. CLOSE1 adopte donc une extension non destructive :

Couche	Role	Statut
pdhg_promoted_v2.py	Solveur lie aux preuves T4 historiques	Immutable
promotion_manifest_v1.json	Liaison des artefacts historiques	Verifie
pdhg_close1.py	Norme exacte + gap + KKT	Nouveau, teste CPU
pdhg_auto_routed_close1.py	API auto-routee avec diagnostics	Nouveau, fallback explicite
run_cuda_validation_close1_colab.py	Provenance pre-run/post-run	Execute; T4 PASS 47/47

10. Reproductibilite

Le PDF contient l'archive ZIP complete comme piece jointe. Apres extraction :

```
python verify_close1_package.py
python -m pytest -q
python run_reference_validation.py
python run_close1_validation.py
```

Sur un Colab Tesla T4, dans le dossier extrait :

```
%run run_cuda_validation_close1_colab.py
```

Controle	Resultat de l'assemblage
Fichiers manifestes	79 verifies
Tests pytest	58 PASS
Oracle NumPy	10 PASS
Manifestes historiques	promotion + routage verifies
Archive ZIP	SHA-256 723e41c0cc78ab7cf180a2a251990b5e bc6b179ba2d46b9976a6271ddab1195c

Controle	Resultat de l'assemblage
Attestation T4 CLOSE1	47/47 PASS; release source-bound

11. Claim defendable et non-claims

Claim defendable : implementation bi-coordonnee de Wright Omega utilisee comme primitive commune de sept prox connus, deployee sur NumPy, PyTorch et Triton, integree a un PDHG-TV periodique fusionne, accompagnee de diagnostics gap/KKT et d'une validation CUDA liee aux sources sur Tesla T4. Le routage de performance reste fonde sur la campagne historique T4 conservee.

- Pas de nouvelle fonction speciale ni de nouveau theoreme proximal.
- Pas de preuve que les prox FP32 fixes satisfont les hypotheses d'un theoreme PDHG inexact global.
- Pas de claim SOTA, production-ready, autograd Triton, AMP, FP16/BF16 ou equivalence inter-GPU.
- L'attestation CLOSE1 ne vaut que pour la Tesla T4, Torch 2.11.0+cu128 et CUDA 12.8 enregistres.

12. Limites et chantier suivant

Question	Connu	Inconnu / a faire
Erreur des prox	Residus elementaires controles	Budget d'erreur sommable le long d'une trajectoire
Gap/KKT	Diagnostics finaux exacts au sens des formules implementees	Borne universelle distance-solution
Performance	T4, deux tailles, cinq modeles	torch.compile, CUDA Graph, RTX 3060, A100
Provenance	Attestation T4 source-bound	Extension a d'autres environnements

CLOSE1 est ferme sur son perimetre T4. Le chantier suivant peut etre une extension controlee a RTX 3060 et A100, avec une attestation et une table de routage distinctes par environnement; aucune extrapolation automatique n'est permise.

Annexe A - Inventaire manifeste

Les hashes complets sont dans close1_package_manifest.json et SHA256SUMS.txt. Le tableau ci-dessous donne un prefixe de controle; l'archive incorporee est l'autorite binaire.

Fichier	SHA-256 (prefixe)
CLOSE1_CLOSURE_REPORT.md	e085b95cc709cb702722
README.md	f228f20d040e2bbc5269
T4_CLOSE1_VALIDATION_STATUS.json	825fb2dc792d653ba588
T4_ROUTING_CLOSURE_REPORT.md	cf744f729b348ff22b24
benchmark_promoted_routing_v1.py	90a416ca254a2d185290
build_close1_manifest.py	781303f8d7ca56a044a7
check_triton_jit_globals_v1.py	2f8a6ecacc25a2659f04
close1_cpu_validation_metrics.json	2d05669fa7ec25ff4a4c
generate_reference_metrics.py	6234675846dcc5844956
historical_reports/BENCHMARK_PROTOCOL_V1.md	3998d25ad885ece0b2f5
historical_reports/BENCHMARK_ROUTING_V1_CLOSURE_REPORT.md	b624e413c74dc3092ae0
historical_reports/PDHG_PROMOTED_V2_CLOSURE_REPORT.md	d5baa4849615cf58c5f4
historical_reports/PDHG_V1_CLOSURE_REPORT.md	5be9c8c5d19db597c618
historical_reports/PILOT_T4_NONSTATISTICAL.md	89d63a290e926d361f70
historical_reports/REFERENCE_V1_CLOSURE_REPORT.md	cda7d53c57972d1edcab
historical_reports/TORCH_V1_CLOSURE_REPORT.md	3fee7108da37dc1b821a
historical_reports/TRITON_PATCH_CLOSURE_REPORT.md	746846a4afbf2775ae17
historical_reports/TRITON_PATCH_V1_1_HOTFIX_REPORT.md	0c25500ec62bade53409
historical_reports/auto_routed_v3_package_manifest.json	0a2b2cf846dd16e0fb18
historical_reports/pdhg_validation_metrics.json	3c574617da66b062c4b2
historical_reports/promotion_integration_metrics.json	814ff24fdf41f4279b4b
historical_reports/reference_metrics.json	d11080f711927891255a
historical_reports/reference_validation_report.json	1a492bb9a82568980242
historical_reports/torch_validation_metrics.json	00dc088374756130d60c
lamBERT_prox_compat_v1.py	d5930aa1b044d3f6996c
lamBERT_prox_reference_v1.py	67fc3df37972d4c152d1
lamBERT_prox_torch_v1.py	7a262b35b66671c6e3e5
merge_routing_reports_v1.py	b1184d2f5a435c37fcfb
pdhg_auto_routed_close1.py	0434e32478a3d8ade91c
pdhg_auto_routed_v3.py	2c4eb14ef7eaa1ab242c
pdhg_canonical_v1.py	6674c6fe95711ae72bdf
pdhg_close1.py	cbd6e31991345a962699
pdhg_promoted_v2.py	b7dd61bffc13fe7db414
pdhg_routing_benchmark_raw_v1.json	ea38a2643f7da8c07916
pdhg_routing_benchmark_summary_v1.md	74883141c33b082bf2bf

Fichier	SHA-256 (prefixe)
pdhg_routing_table_v1.json	e4e21b82175d53d76e35
pdhg_validation_metrics.json	cfc5106c145479c48b8e
production_route_preview_v1.json	176ba05f7ec91ec325aa
promotion_manifest_v1.json	a463aa317d8364fea15b
reference_validation_report.json	a70c6bcc3ff526735850
requirements.txt	26ff947618353810126b
routing_evidence_manifest_v1.json	65ccf3cb925315fbb02b
routing_policy_v1.py	073ad67fcfec125cb783
routing_policy_v2.py	a7da6295e370c66e288a
run_auto_routing_smoke_colab.py	bf2ebb95b69ac188b014
run_benchmark_ci_v1.py	72921d7c0ca02b90efd9
run_close1_validation.py	73cac86e98bd9923b960
run_cuda_validation_close1_colab.py	67ed58aeea42e394c0c2
run_cuda_validation_colab.py	388b0b8583a0141b10c8
run_pdhg_validation.py	3f5b6702eaf3104919c5
run_promoted_regression_colab.py	89ce2dafb6eac3cc4e41
run_reference_validation.py	84e194071d867d74d1aa
run_torch_validation.py	8280cca43605a7c5a2ed
static_audit_triton_candidates_v1.py	b97ef19d774d9ff5a69b
t4_close1_colab_stdout_2026-07-31.txt	839f0fa042dbd6175e88
t4_routing_closure_metrics_v1.json	2503b37526224d3d69c9
test_lambert_prox_reference_v1.py	fc4634610b7b37db2236
test_lambert_prox_torch_v1.py	ecfd04ac3342b5d87304
test_pdhg_auto_routed_close1.py	e947bdd1f8cae48a4743
test_pdhg_auto_routed_v3_cpu.py	45122938003fff0caa25
test_pdhg_canonical_v1.py	7197f3c06a9aa4653945
test_pdhg_close1.py	a10b38cbbd0e42a0f05a
test_pdhg_promoted_v2_cpu.py	9516a10cd747a2cf3def
test_routing_policy_v1.py	42de767060f3629eb4b5
test_routing_policy_v2.py	7828b20a4f909c59f62f
torch_validation_metrics.json	e47cc364db3e53a40f63
triton_oracle_validation_v1.py	cbae02104bc80f4b91b5
triton_oracle_validation_v2.py	03e1aee20e4c1cab2dea
triton_promotion_gate_v1.py	a17047c671fe2dddd672
triton_prox_canonical_v1.py	12ab8ddb87619f91283c
triton_prox_fused_candidate_v2.py	adc15b8ea82ec28591e9
triton_static_audit.json	b618afc621943d362ce2
triton_stencil_candidate_v5.py	eb529f098d50f6311e82

Fichier	SHA-256 (prefixe)
triton_stencil_canonical_v1.py	cacb17459bdb25f01fa7
triton_validation_metrics.json	f75176058482fcdba225
triton_validation_metrics_close1.json	76b4aec4c5aa89079ce2
triton_validation_metrics_v2.json	cb2a5a44aa05f7e923ea
verify_close1_package.py	6573725511dc05ba51a7
verify_promoted_package.py	0cbc49d2bc5fdf34c097

Annexe B - Sources CLOSE1

Les listings suivants reproduisent les fichiers qui portent les corrections. Les dependances historiques et les tests supplementaires sont tous presents dans la piece jointe ZIP.

Source complete - pdhg_close1.py

Transcription de lecture. Les lignes longues sont repliees visuellement; les octets normatifs sont ceux de l'archive ZIP incorporee.

```

0001 | """CLOSE1 certification layer for the promoted Lambert-Prox PDHG solver.
0002 |
0003 | The promoted V2 solver and the T4 evidence bound to its byte hash are kept
0004 | immutable. This module calls that solver, supplies the exact finite-grid norm
0005 | when possible, and computes independent end-of-run diagnostics:
0006 |
0007 | * relative primal and dual iterate changes (stabilization only),
0008 | * primal and dual objectives and their Fenchel gap,
0009 | * normalized primal and dual resolvent/KKT residuals,
0010 | * pointwise dual-feasibility violation.
0011 |
0012 | ``certified`` is a numerical acceptance flag for user-selected tolerances. It
0013 | is not a proof of convergence of finite precision or inexact-prox iterations.
0014 | """
0015 | from __future__ import annotations
0016 |
0017 | from dataclasses import asdict, dataclass
0018 | import math
0019 | import time
0020 | from typing import Any
0021 |
0022 | import torch
0023 |
0024 | import pdhg_promoted_v2 as legacy
0025 |
0026 |
0027 | @dataclass(frozen=True)
0028 | class Close1Diagnostics:
0029 |     primal_objective: float
0030 |     dual_objective: float
0031 |     primal_dual_gap: float
0032 |     relative_gap: float
0033 |     kkt_primal_residual: float
0034 |     kkt_dual_residual: float
0035 |     kkt_max_residual: float
0036 |     dual_feasibility_violation: float
0037 |     finite: bool
0038 |
0039 |     def to_dict(self) -> dict[str, Any]:
0040 |         return asdict(self)
0041 |
0042 |
0043 | @dataclass(frozen=True)
0044 | class PDHGInfoClose1:
0045 |     iterations: int
0046 |     stabilized: bool
0047 |     certified: bool
0048 |     stop_reason: str
0049 |     tau: float
0050 |     sigma: float
0051 |     norm_k_exact: float
0052 |     norm_k_step_bound: float
0053 |     backend: legacy.BackendReport
0054 |     primal_rel_change: float
0055 |
0056 |     dual_rel_change: float
0057 |     diagnostics: Close1Diagnostics
0058 |     solver_elapsed_seconds: float
0059 |     diagnostics_elapsed_seconds: float
0060 |     history: dict[str, list[float]]
0061 |     status_note: str
0062 |
0063 |     def to_dict(self) -> dict[str, Any]:
0064 |         return asdict(self)
0065 |
0066 |
0067 | def norm_k_upper_bound_periodic_2d() -> float:
0068 |     """Dimension-independent safe bound for the periodic forward gradient."""
0069 |     return math.sqrt(8.0)
0070 |
0071 |
0072 | def norm_k_periodic_2d(height: int, width: int) -> float:
0073 |     """Exact l2 operator norm of the periodic 2-D forward-difference gradient.
0074 |
0075 |     For a H x W grid, Fourier diagonalization gives
0076 |
0077 |         ||K||^2 = 4 sin^2(pi floor(W/2)/W)
0078 |                 + 4 sin^2(pi floor(H/2)/H).
0079 |
0080 |     The value is sqrt(8) when both dimensions are even and is strictly smaller
0081 |     when at least one nontrivial dimension is odd.
0082 |     """
0083 |     if not isinstance(height, int) or not isinstance(width, int):
0084 |         raise TypeError("height and width must be integers")

```

```

0084 |     if height <= 0 or width <= 0:
0085 |         raise ValueError("height and width must be positive")
0086 |     wx = 4.0 * math.sin(math.pi * (width // 2) / width) ** 2
0087 |     wy = 4.0 * math.sin(math.pi * (height // 2) / height) ** 2
0088 |     return math.sqrt(wx + wy)
0089 |
0090 |
0091 | def fidelity_value_closed(
0092 |     kind: str,
0093 |     x: torch.Tensor,
0094 |     y_obs: torch.Tensor | None,
0095 | ) -> torch.Tensor:
0096 |     """Lower-semicontinuous closed fidelity used for primal/dual diagnostics."""
0097 |     k = kind.lower().strip()
0098 |     inf = torch.full((), float("inf"), dtype=x.dtype, device=x.device)
0099 |     if k in ("gaussian", "l2"):
0100 |         if y_obs is None:
0101 |             raise ValueError("gaussian requires y_obs")
0102 |         return 0.5 * ((x - y_obs) ** 2).sum()
0103 |     if k == "poisson_intensity":
0104 |         if y_obs is None or bool((y_obs < 0).any().item()):
0105 |             raise ValueError("poisson_intensity requires y_obs >= 0")
0106 |         if bool((x < 0).any().item()) or bool(((x == 0) & (y_obs > 0)).any().item()):
0107 |             return inf
0108 |         positive_y = y_obs > 0
0109 |
0110 |         log_x = torch.where(positive_y, torch.log(x), torch.zeros_like(x))
0111 |         return (x - torch.where(positive_y, y_obs * log_x, torch.zeros_like(x))).sum()
0112 |     if k == "poisson_log":
0113 |         if y_obs is None:
0114 |             raise ValueError("poisson_log requires y_obs")
0115 |         return (torch.exp(x) - y_obs * x).sum()
0116 |     if k == "kl":
0117 |         if y_obs is None or bool((y_obs <= 0).any().item()):
0118 |             raise ValueError("KL requires y_obs > 0")
0119 |         if bool((x < 0).any().item()):
0120 |             return inf
0121 |         positive_x = x > 0
0122 |         term = torch.where(
0123 |             positive_x,
0124 |             x * (torch.log(x) - torch.log(y_obs)) - x + y_obs,
0125 |             y_obs,
0126 |         )
0127 |         return term.sum()
0128 |     if k == "xlogx":
0129 |         if bool((x < 0).any().item()):
0130 |             return inf
0131 |         return torch.where(x > 0, x * torch.log(x), torch.zeros_like(x)).sum()
0132 |     if k == "exp":
0133 |         return torch.exp(x).sum()
0134 |     if k == "neglog":
0135 |         if bool((x <= 0).any().item()):
0136 |             return inf
0137 |         return (-torch.log(x)).sum()
0138 |     raise ValueError(f"unsupported fidelity kind: {kind!r}")
0139 |
0140 | def fidelity_conjugate_value(
0141 |     kind: str,
0142 |     s: torch.Tensor,
0143 |     y_obs: torch.Tensor | None,
0144 | ) -> torch.Tensor:
0145 |     """Value of F*(s) for the five well-posed PDHG fidelities."""
0146 |     k = kind.lower().strip()
0147 |     inf = torch.full((), float("inf"), dtype=s.dtype, device=s.device)
0148 |     if k in ("gaussian", "l2"):
0149 |         if y_obs is None:
0150 |             raise ValueError("gaussian requires y_obs")
0151 |         return (0.5 * s * s + s * y_obs).sum()
0152 |     if k == "poisson_intensity":
0153 |         if y_obs is None or bool((y_obs < 0).any().item()):
0154 |             raise ValueError("poisson_intensity requires y_obs >= 0")
0155 |         if bool((s > 1).any().item()) or bool(((s == 1) & (y_obs > 0)).any().item()):
0156 |             return inf
0157 |         positive_y = y_obs > 0
0158 |         term = torch.where(
0159 |             positive_y,
0160 |             y_obs * (torch.log(y_obs) - torch.log1p(-s) - 1.0),
0161 |             torch.zeros_like(s),
0162 |         )
0163 |         return term.sum()
0164 |     if k == "poisson_log":
0165 |         if y_obs is None:
0166 |             raise ValueError("poisson_log requires y_obs")
0167 |         t = s + y_obs
0168 |         if bool((t < 0).any().item()):
0169 |             return inf
0170 |         return torch.where(t > 0, t * torch.log(t) - t, torch.zeros_like(t)).sum()
0171 |     if k == "kl":
0172 |         if y_obs is None or bool((y_obs <= 0).any().item()):
0173 |             raise ValueError("KL requires y_obs > 0")
0174 |         return (y_obs * (torch.exp(s) - 1.0)).sum()
0175 |     if k == "xlogx":
0176 |         return torch.exp(s - 1.0).sum()
0177 |     raise ValueError(f"no finite-model conjugate implemented for {kind!r}")
0178 |
0179 |
0180 | def primal_objective(

```

```

0181 |     kind: str,
0182 |     x: torch.Tensor,
0183 |     y_obs: torch.Tensor | None,
0184 |     lam_tv: float,
0185 | ) -> torch.Tensor:
0186 |     return fidelity_value_closed(kind, x, y_obs) + float(lam_tv) * legacy.tv_iso(x)
0187 |
0188 |
0189 | def diagnose_solution(
0190 |     kind: str,
0191 |     x: torch.Tensor,
0192 |     dual: torch.Tensor,
0193 |     y_obs: torch.Tensor | None,
0194 |     lam_tv: float,
0195 |     *,
0196 |     tau: float,
0197 |     sigma: float,
0198 | ) -> CloselDiagnostics:
0199 |     """Compute gap and normalized resolvent residuals at one PDHG state."""
0200 |     if tau <= 0 or sigma <= 0:
0201 |         raise ValueError("tau and sigma must be positive")
0202 |     with torch.no_grad():
0203 |         kx = legacy.K(x)
0204 |         ktp = legacy.Kt(dual)
0205 |         dual_norm = torch.sqrt(dual[:, :1] ** 2 + dual[:, 1:] ** 2)
0206 |         dual_violation_t = torch.clamp(dual_norm - float(lam_tv), min=0.0).max()
0207 |
0208 |         x_fixed = legacy.canonical_prox_f(kind, x - float(tau) * ktp, float(tau), y_obs)
0209 |         dual_fixed = legacy.prox_gstar_tv_iso(
0210 |             dual + float(sigma) * kx, float(lam_tv)
0211 |         )
0212 |         rx = (x - x_fixed).norm() / float(tau)
0213 |         rp = (dual - dual_fixed).norm() / float(sigma)
0214 |         rx_scale = max(1.0, float(x.norm().item()), float(ktp.norm().item()))
0215 |         rp_scale = max(1.0, float(dual.norm().item()), float(kx.norm().item()))
0216 |         rx_n = float(rx.item()) / rx_scale
0217 |
0218 |         rp_n = float(rp.item()) / rp_scale
0219 |
0220 |         primal_t = primal_objective(kind, x, y_obs, lam_tv)
0221 |         fstar_t = fidelity_conjugate_value(kind, -ktp, y_obs)
0222 |         primal = float(primal_t.item())
0223 |         feasible = float(dual_violation_t.item()) <= 16.0 * torch.finfo(x.dtype).eps
0224 |         dual_obj = -float(fstar_t.item()) if feasible else float("-inf")
0225 |         gap = primal - dual_obj
0226 |         scale = max(1.0, abs(primal), abs(dual_obj)) if math.isfinite(dual_obj) else float("inf")
0227 |         rel_gap = abs(gap) / scale if math.isfinite(gap) and math.isfinite(scale) else float("inf")
0228 |         finite = all(math.isfinite(v) for v in (primal, dual_obj, gap, rel_gap, rx_n, rp_n))
0229 |         return CloselDiagnostics(
0230 |             primal_objective=primal,
0231 |             dual_objective=dual_obj,
0232 |             primal_dual_gap=gap,
0233 |             relative_gap=rel_gap,
0234 |             kkt_primal_residual=rx_n,
0235 |             kkt_dual_residual=rp_n,
0236 |             kkt_max_residual=max(rx_n, rp_n),
0237 |             dual_feasibility_violation=float(dual_violation_t.item()),
0238 |             finite=finite,
0239 |         )
0240 |
0241 | def pdhg(
0242 |     kind: str,
0243 |     x0: torch.Tensor,
0244 |     y_obs: torch.Tensor | None,
0245 |     lam_tv: float,
0246 |     *,
0247 |     kkt_tol: float = 1.0e-5,
0248 |     gap_tol: float = 1.0e-5,
0249 |     norm_k: float | None = None,
0250 |     **kwargs: Any,
0251 | ) -> tuple[torch.Tensor, torch.Tensor, PDHGInfoClosel]:
0252 |     """Run the immutable promoted solver and attach CLOSE1 diagnostics."""
0253 |     if x0.ndim != 4:
0254 |         raise ValueError("x0 must be a 4-D tensor")
0255 |     if kkt_tol < 0 or gap_tol < 0 or not math.isfinite(kkt_tol + gap_tol):
0256 |         raise ValueError("kkt_tol and gap_tol must be finite and nonnegative")
0257 |     exact_norm = norm_k_periodic_2d(int(x0.shape[-2]), int(x0.shape[-1]))
0258 |     if norm_k is None:
0259 |         step_bound = exact_norm if exact_norm > 0.0 else norm_k_upper_bound_periodic_2d()
0260 |     else:
0261 |         step_bound = float(norm_k)
0262 |         slack = 64.0 * math.ulp(max(1.0, exact_norm))
0263 |         if step_bound + slack < exact_norm:
0264 |             raise ValueError(
0265 |                 f"norm_k={step_bound} underestimates the exact finite-grid norm {exact_norm}"
0266 |             )
0267 |
0268 |     x, dual, old = legacy.pdhg(
0269 |         kind, x0, y_obs, lam_tv, norm_k=step_bound, **kwargs
0270 |     )
0271 |
0272 |     t0 = time.perf_counter()
0273 |     diagnostics = diagnose_solution(
0274 |         kind, x, dual, y_obs, lam_tv, tau=old.tau, sigma=old.sigma
0275 |     )
0276 |     diagnostics_elapsed = time.perf_counter() - t0
0277 |     certified = bool(
0278 |         diagnostics.finite

```

```

0278 |         and diagnostics.kkt_max_residual <= kkt_tol
0279 |         and diagnostics.relative_gap <= gap_tol
0280 |         and diagnostics.dual_feasibility_violation <= kkt_tol
0281 |     )
0282 |     stabilized = bool(old.converged)
0283 |     stop_reason = (
0284 |         "relative-change tolerance met" if stabilized else "maximum iterations reached"
0285 |     )
0286 |     history = {key: list(value) for key, value in old.history.items()}
0287 |     history.update({
0288 |         "final_relative_gap": [diagnostics.relative_gap],
0289 |         "final_kkt_primal_residual": [diagnostics.kkt_primal_residual],
0290 |         "final_kkt_dual_residual": [diagnostics.kkt_dual_residual],
0291 |     })
0292 |     info = PDHGInfoClose1(
0293 |         iterations=old.iterations,
0294 |         stabilized=stabilized,
0295 |         certified=certified,
0296 |         stop_reason=stop_reason,
0297 |         tau=old.tau,
0298 |         sigma=old.sigma,
0299 |         norm_k_exact=exact_norm,
0300 |         norm_k_step_bound=step_bound,
0301 |         backend=old.backend,
0302 |         primal_rel_change=old.primal_rel_change,
0303 |         dual_rel_change=old.dual_rel_change,
0304 |         diagnostics=diagnostics,
0305 |         solver_elapsed_seconds=old.elapsed_seconds,
0306 |         diagnostics_elapsed_seconds=diagnostics_elapsed,
0307 |         history=history,
0308 |         status_note=(
0309 |             "Numerical diagnostic thresholds passed; this is not a floating-point "
0310 |             "convergence proof." if certified else
0311 |             "End state is not certified at the requested KKT/gap tolerances."
0312 |         ),
0313 |     )
0314 |     return x, dual, info
0315 |
0316 |
0317 | __all__ = [
0318 |     "Close1Diagnostics", "PDHGInfoClose1",
0319 |     "norm_k_upper_bound_periodic_2d", "norm_k_periodic_2d",
0320 |     "fidelity_value_closed", "fidelity_conjugate_value", "primal_objective",
0321 |     "diagnose_solution", "pdhg",
0322 | ]

```

Source complete - pdhg_auto_routed_close1.py

Transcription de lecture. Les lignes longues sont repliées visuellement; les octets normatifs sont ceux de l'archive ZIP incorporée.

```

0001 | """Evidence-backed auto routing with CLOSE1 diagnostics."""
0002 | from __future__ import annotations
0003 |
0004 | from dataclasses import asdict, dataclass
0005 | from pathlib import Path
0006 | from typing import Any
0007 |
0008 | import torch
0009 |
0010 | import pdhg_close1
0011 | import pdhg_promoted_v2 as legacy
0012 | from routing_policy_v2 import (
0013 |     DEFAULT_EVIDENCE_MANIFEST,
0014 |     DEFAULT_TABLE,
0015 |     RouteDecision,
0016 |     RoutingEvidenceError,
0017 |     select_backend,
0018 | )
0019 |
0020 |
0021 | @dataclass(frozen=True)
0022 | class AutoRoutingReportClose1:
0023 |     mode: str
0024 |     selected_backend: str
0025 |     executed_backend: str
0026 |     fallback_used: bool
0027 |     fallback_reason: str | None
0028 |     decision: dict[str, Any]
0029 |
0030 |     def to_dict(self) -> dict[str, Any]:
0031 |         return asdict(self)
0032 |
0033 |
0034 | def pdhg_auto(
0035 |     kind: str,
0036 |     x0: torch.Tensor,
0037 |     y_obs: torch.Tensor | None,
0038 |     lam_tv: float,
0039 |     *,
0040 |     backend: str = "auto",
0041 |     routing_table: str | Path = DEFAULT_TABLE,
0042 |     routing_evidence_manifest: str | Path = DEFAULT_EVIDENCE_MANIFEST,
0043 |     max_area_ratio: float | None = None,
0044 |     max_aspect_ratio_factor: float = 2.0,
0045 |     safe_auto_fallback: bool = True,
0046 |     **pdhg_kwargs: Any,
0047 | ):

```

```

0048 | mode = str(backend).lower().strip()
0049 | if mode != "auto":
0050 |     selected = legacy.Backend(mode)
0051 |     x, dual, info = pdhg_close1.pdhg(
0052 |         kind, x0, y_obs, lam_tv, backend=selected, **pdhg_kwargs
0053 |     )
0054 |     decision = RouteDecision(
0055 |         requested=mode, selected_backend=mode, reason="explicit backend request",
0056 |         environment_key="explicit", kind=kind.lower().strip(),
0057 |         height=int(x0.shape[-2]), width=int(x0.shape[-1]),
0058 |     )
0059 |     report = AutoRoutingReportClose1(
0060 |         mode, mode, info.backend.executed, False, None, decision.to_dict()
0061 |     )
0062 |     return x, dual, info, report
0063 |
0064 |     try:
0065 |         decision = select_backend(
0066 |             kind, int(x0.shape[-2]), int(x0.shape[-1]),
0067 |             device=x0.device, dtype=x0.dtype,
0068 |             table_path=routing_table,
0069 |             evidence_manifest=routing_evidence_manifest,
0070 |             max_area_ratio=max_area_ratio,
0071 |             max_aspect_ratio_factor=max_aspect_ratio_factor,
0072 |         )
0073 |     except RoutingEvidenceError as exc:
0074 |         if not safe_auto_fallback:
0075 |             raise
0076 |         decision = RouteDecision(
0077 |             requested="auto", selected_backend="torch",
0078 |             reason=f"routing evidence rejected: {exc}", environment_key="unverified",
0079 |             kind=kind.lower().strip(), height=int(x0.shape[-2]),
0080 |             width=int(x0.shape[-1]), evidence_verified=False,
0081 |         )
0082 |
0083 |     chosen = decision.selected_backend
0084 |     try:
0085 |         x, dual, info = pdhg_close1.pdhg(
0086 |             kind, x0, y_obs, lam_tv, backend=chosen, **pdhg_kwargs
0087 |         )
0088 |         report = AutoRoutingReportClose1(
0089 |             "auto", chosen, info.backend.executed, False, None, decision.to_dict()
0090 |         )
0091 |         return x, dual, info, report
0092 |     except (legacy.BackendUnavailableError, legacy.PromotionEvidenceError, ImportError) as exc:
0093 |         if chosen == "torch" or not safe_auto_fallback:
0094 |             raise
0095 |         x, dual, info = pdhg_close1.pdhg(
0096 |             kind, x0, y_obs, lam_tv, backend="torch", **pdhg_kwargs
0097 |         )
0098 |         report = AutoRoutingReportClose1(
0099 |             "auto", chosen, "torch", True,
0100 |             f"accelerated backend rejected at execution: {exc}", decision.to_dict(),
0101 |         )
0102 |         return x, dual, info, report
0103 |
0104 |
0105 | __all__ = ["AutoRoutingReportClose1", "pdhg_auto"]

```

Source complete - run_cuda_validation_close1_colab.py

Transcription de lecture. Les lignes longues sont repliées visuellement; les octets normatifs sont ceux de l'archive ZIP incorporée.

```

0001 | """No-argument T4 validation with source hashes captured before CUDA work.
0002 |
0003 | Run this file from the extracted CLOSE1 directory in a Tesla T4 Colab. The
0004 | script hashes every source involved in the oracle comparison before importing
0005 | the validation module, executes the 47 comparisons, hashes the sources again,
0006 | and accepts PASS only when the two maps are identical.
0007 | """
0008 | from __future__ import annotations
0009 |
0010 | from datetime import datetime, timezone
0011 | import hashlib
0012 | import importlib
0013 | import json
0014 | from pathlib import Path
0015 |
0016 |
0017 | HERE = Path(__file__).resolve().parent
0018 | PRE_RUN = HERE / "pre_run_provenance_close1.json"
0019 | REPORT = HERE / "triton_validation_metrics_close1.json"
0020 | SOURCES = (
0021 |     "lamBERT_prox_torch_v1.py",
0022 |     "pdhg_canonical_v1.py",
0023 |     "pdhg_promoted_v2.py",
0024 |     "triton_prox_canonical_v1.py",
0025 |     "triton_stencil_canonical_v1.py",
0026 |     "triton_oracle_validation_v2.py",
0027 |     "triton_promotion_gate_v1.py",
0028 |     "run_cuda_validation_close1_colab.py",
0029 | )
0030 |
0031 |
0032 | def sha256_file(path: Path) -> str:
0033 |     h = hashlib.sha256()

```

```

0034 |     with path.open("rb") as stream:
0035 |         for block in iter(lambda: stream.read(1 << 20), b""):
0036 |             h.update(block)
0037 |     return h.hexdigest()
0038 |
0039 |
0040 | def source_map() -> dict[str, str]:
0041 |     missing = [name for name in SOURCES if not (HERE / name).is_file()]
0042 |     if missing:
0043 |         raise FileNotFoundError(f"missing CLOSE1 validation sources: {missing}")
0044 |     return {name: sha256_file(HERE / name) for name in SOURCES}
0045 |
0046 |
0047 | def release_digest(hashes: dict[str, str]) -> str:
0048 |     payload = json.dumps(hashes, sort_keys=True, separators=(",", ":")).encode("utf-8")
0049 |     return hashlib.sha256(payload).hexdigest()
0050 |
0051 |
0052 | def main() -> int:
0053 |     before = source_map()
0054 |     begun = datetime.now(timezone.utc).isoformat()
0055 |
0056 |     pre = {
0057 |         "schema": "lamBERT-prox-close1-pre-run-provenance-v1",
0058 |         "captured_before_validation_import": True,
0059 |         "started_at_utc": begun,
0060 |         "release_sha256": release_digest(before),
0061 |         "sources": before,
0062 |     }
0063 |     PRE_RUN.write_text(json.dumps(pre, indent=2), encoding="utf-8")
0064 |
0065 |     validator = importlib.import_module("triton_oracle_validation_v2")
0066 |     inner_path = HERE / "triton_validation_metrics_v2_close1_inner.json"
0067 |     inner = validator.run(inner_path)
0068 |     after = source_map()
0069 |     unchanged = before == after
0070 |     status = "PASS" if inner.get("status") == "PASS" and unchanged else inner.get("status", "FAIL")
0071 |     if inner.get("status") == "PASS" and not unchanged:
0072 |         status = "FAIL"
0073 |     report = {
0074 |         "schema": "lamBERT-prox-triton-validation-close1-v1",
0075 |         "status": status,
0076 |         "started_at_utc": begun,
0077 |         "finished_at_utc": datetime.now(timezone.utc).isoformat(),
0078 |         "provenance": {
0079 |             "level": "pre_run_and_post_run_source_binding",
0080 |             "release_sha256": release_digest(before),
0081 |             "source_hashes_before": before,
0082 |             "source_hashes_after": after,
0083 |             "sources_unchanged_during_run": unchanged,
0084 |         },
0085 |         "validation": inner,
0086 |         "interpretation": (
0087 |             "PASS binds this exact source map to this execution. It does not extend "
0088 |             "the device claim beyond the environment recorded by the inner report."
0089 |         ),
0090 |     }
0091 |     REPORT.write_text(json.dumps(report, indent=2), encoding="utf-8")
0092 |     print(json.dumps(report, indent=2))
0093 |     return 0 if status == "PASS" else (3 if status == "NOT_RUN" else 1)
0094 |
0095 | if __name__ == "__main__":
0096 |     raise SystemExit(main())

```

Source complete - test_pdhg_close1.py

Transcription de lecture. Les lignes longues sont repliées visuellement; les octets normatifs sont ceux de l'archive ZIP incorporée.

```

0001 | import math
0002 |
0003 | import pytest
0004 | import torch
0005 |
0006 | from pdhg_close1 import (
0007 |     diagnose_solution,
0008 |     fidelity_conjugate_value,
0009 |     fidelity_value_closed,
0010 |     norm_k_periodic_2d,
0011 |     norm_k_upper_bound_periodic_2d,
0012 |     pdhg,
0013 | )
0014 |
0015 |
0016 | def test_finite_grid_norm_even_and_odd():
0017 |     assert norm_k_periodic_2d(32, 48) == pytest.approx(math.sqrt(8.0), rel=0, abs=1e-15)
0018 |     odd = norm_k_periodic_2d(31, 47)
0019 |     expected = math.sqrt(
0020 |         4 * math.sin(math.pi * 23 / 47) ** 2
0021 |         + 4 * math.sin(math.pi * 15 / 31) ** 2
0022 |     )
0023 |     assert odd == pytest.approx(expected, rel=0, abs=1e-15)
0024 |     assert odd < norm_k_upper_bound_periodic_2d()
0025 |
0026 |
0027 | @pytest.mark.parametrize("kind", [
0028 |     "gaussian", "poisson_intensity", "poisson_log", "kl", "xlogx",

```

```

0029 | })
0030 | def test_fenchel_young_equality_at_gradient(kind):
0031 |     dtype = torch.float64
0032 |     x = torch.tensor([[[[0.7, 1.3], [2.0, 0.4]]]], dtype=dtype)
0033 |     obs = torch.tensor([[[[0.8, 1.1], [1.4, 0.6]]]], dtype=dtype)
0034 |     if kind == "gaussian":
0035 |         s, y = x - obs, obs
0036 |     elif kind == "poisson_intensity":
0037 |         s, y = 1.0 - obs / x, obs
0038 |     elif kind == "poisson_log":
0039 |         s, y = torch.exp(x) - obs, obs
0040 |     elif kind == "kl":
0041 |         s, y = torch.log(x / obs), obs
0042 |     else:
0043 |         s, y = torch.log(x) + 1.0, None
0044 |     lhs = fidelity_value_equality_at_gradient(kind, x, y) + fidelity_conjugate_value(kind, s, y)
0045 |     rhs = (x * s).sum()
0046 |     torch.testing.assert_close(lhs, rhs, rtol=2e-14, atol=2e-14)
0047 |
0048 |
0049 | def test_constant_gaussian_solution_is_certified():
0050 |     obs = torch.full((1, 1, 9, 11), 1.25, dtype=torch.float64)
0051 |     x, dual, info = pdhg(
0052 |         "gaussian", obs.clone(), obs, 0.08,
0053 |         max_iter=30, min_iter=2, tol=1e-12, backend="torch", monitor_every=2,
0054 |         kkt_tol=1e-12, gap_tol=1e-12,
0055 |     )
0056 |     assert info.stabilized
0057 |     assert info.certified
0058 |     assert info.diagnostics.kkt_max_residual <= 1e-12
0059 |     assert info.diagnostics.relative_gap <= 1e-12
0060 |     assert info.norm_k_exact < math.sqrt(8.0)
0061 |     torch.testing.assert_close(x, obs, rtol=0, atol=0)
0062 |     torch.testing.assert_close(dual, torch.zeros_like(dual), rtol=0, atol=0)
0063 |
0064 |
0065 | def test_underestimated_norm_is_rejected():
0066 |     obs = torch.ones((1, 1, 8, 10), dtype=torch.float64)
0067 |     with pytest.raises(ValueError, match="underestimates"):
0068 |         pdhg("gaussian", obs, obs, 0.1, norm_k=2.0, max_iter=1, min_iter=0)
0069 |
0070 |
0071 | def test_gap_is_nonnegative_up_to_roundoff():
0072 |     obs = torch.rand((1, 1, 7, 9), dtype=torch.float64) + 0.2
0073 |     x, dual, info = pdhg(
0074 |         "gaussian", obs.clone(), obs, 0.05,
0075 |         max_iter=80, min_iter=80, tol=0.0, backend="torch",
0076 |     )
0077 |     d = diagnose_solution(
0078 |         "gaussian", x, dual, obs, 0.05, tau=info.tau, sigma=info.sigma
0079 |     )
0080 |     assert d.finite
0081 |     assert d.primal_dual_gap >= -1e-10

```

Source complete - verify_close1_package.py

Transcription de lecture. Les lignes longues sont repliées visuellement; les octets normatifs sont ceux de l'archive ZIP incorporée.

```

0001 | """Verify the autonomous CLOSE1 package without importing Torch or Triton."""
0002 | from __future__ import annotations
0003 |
0004 | import hashlib
0005 | import json
0006 | from pathlib import Path
0007 |
0008 |
0009 | HERE = Path(__file__).resolve().parent
0010 | MANIFEST = HERE / "close1_package_manifest.json"
0011 |
0012 |
0013 | def sha256_file(path: Path) -> str:
0014 |     h = hashlib.sha256()
0015 |     with path.open("rb") as stream:
0016 |         for block in iter(lambda: stream.read(1 << 20), b''):
0017 |             h.update(block)
0018 |     return h.hexdigest()
0019 |
0020 |
0021 | def verify_hash_map(base: Path, files: dict[str, str]) -> list[str]:
0022 |     errors = []
0023 |     for name, expected in files.items():
0024 |         path = base / name
0025 |         if not path.is_file():
0026 |             errors.append(f"missing: {name}")
0027 |         elif sha256_file(path) != expected:
0028 |             errors.append(f"hash mismatch: {name}")
0029 |     return errors
0030 |
0031 |
0032 | def release_digest(hashes: dict[str, str]) -> str:
0033 |     payload = json.dumps(hashes, sort_keys=True, separators=(",", ":")).encode("utf-8")
0034 |     return hashlib.sha256(payload).hexdigest()
0035 |
0036 |
0037 | def main() -> int:
0038 |     if not MANIFEST.is_file():

```



```

0039 |         print("FAIL: closel_package_manifest.json is missing")
0040 |         return 1
0041 |     data = json.loads(MANIFEST.read_text(encoding="utf-8"))
0042 |     if data.get("schema") != "lamBERT-prox-closel-package-v2":
0043 |         print("FAIL: unexpected CLOSE1 manifest schema")
0044 |         return 1
0045 |     errors = verify_hash_map(HERE, data.get("files", {}))
0046 |
0047 |     t4_path = HERE / "triton_validation_metrics_closel.json"
0048 |     if not t4_path.is_file():
0049 |         errors.append("missing CLOSE1 T4 attestation")
0050 |     else:
0051 |         t4 = json.loads(t4_path.read_text(encoding="utf-8"))
0052 |         provenance = t4.get("provenance", {})
0053 |         before = provenance.get("source_hashes_before", {})
0054 |         after = provenance.get("source_hashes_after", {})
0055 |
0056 |         summary = t4.get("validation", {}).get("summary", {})
0057 |         if t4.get("status") != "PASS":
0058 |             errors.append("CLOSE1 T4 attestation is not PASS")
0059 |         if before != after or not provenance.get("sources_unchanged_during_run"):
0060 |             errors.append("CLOSE1 T4 source maps differ")
0061 |         errors.extend(f"T4 source map: {error}" for error in verify_hash_map(HERE, before))
0062 |         if release_digest(before) != provenance.get("release_sha256"):
0063 |             errors.append("CLOSE1 T4 release digest mismatch")
0064 |         if summary.get("count") != 47 or summary.get("passed") != 47:
0065 |             errors.append("CLOSE1 T4 summary is not 47/47")
0066 |         if t4.get("validation", {}).get("environment", {}).get("device") != "Tesla T4":
0067 |             errors.append("CLOSE1 T4 device is not Tesla T4")
0068 |
0069 |     historical = {
0070 |         "promotion_manifest_v1.json": "sources",
0071 |         "routing_evidence_manifest_v1.json": "files",
0072 |     }
0073 |     for manifest_name, field in historical.items():
0074 |         path = HERE / manifest_name
0075 |         if not path.is_file():
0076 |             errors.append(f"missing historical manifest: {manifest_name}")
0077 |             continue
0078 |         item = json.loads(path.read_text(encoding="utf-8"))
0079 |         errors.extend(
0080 |             f"{manifest_name}: {error}"
0081 |             for error in verify_hash_map(HERE, item.get(field, {}))
0082 |         )
0083 |
0084 |     if errors:
0085 |         print(json.dumps({"status": "FAIL", "errors": errors}, indent=2))
0086 |         return 1
0087 |     print(json.dumps({
0088 |         "status": "PASS",
0089 |         "closel_files_verified": len(data["files"]),
0090 |         "historical_manifests_checked": len(historical),
0091 |         "t4_closel_attestation": "PASS",
0092 |     }, indent=2))
0093 |     return 0
0094 |
0095 | if __name__ == "__main__":
0096 |     raise SystemExit(main())

```

References

- A. Chambolle et T. Pock, A First-Order Primal-Dual Algorithm for Convex Problems with Applications to Imaging, Journal of Mathematical Imaging and Vision 40 (2011), DOI 10.1007/s10851-010-0251-1.
- R. M. Corless et D. J. Jeffrey, The Wright Omega Function, dans Artificial Intelligence, Automated Reasoning, and Symbolic Computation (2002).
- J. Rasch et A. Chambolle, Inexact First-Order Primal-Dual Algorithms, Computational Optimization and Applications (2020), arXiv:1803.10576.
- M. El Gheche, G. Chierchia et J.-C. Pesquet, Proximity Operators of Discrete Information Divergences, IEEE Transactions on Information Theory 64 (2018), arXiv:1606.09552.

Fin de la formalisation. Piece jointe normative :
LAMBERT_PROX_CLOSE1_AUTONOME_2026-07-31.zip.